`whitenoise v0.2.0 · Python Package`

Stochastic White Noise Analysis with Memory

Workshop Hands-on Guide

Date: March 14, 2026

Package: `whitenoise` v0.2.0

Authors: Bernido Group, University of San Carlos

Framework: Hida White Noise Calculus · Bernido & Carpio-Bernido (2015)

> *Stochastic White Noise Analysis (SWNA) is a framework for characterizing the memory properties of fluctuating observables in physical and biological systems. Rooted in Hida's white noise calculus, the approach developed by Bernido & Carpio-Bernido (University of San Carlos / CVIF, Philippines) represents a system's trajectory as a stochastic integral* $x(T) = x_0 + \int_0^T f(T-t) \cdot h(t) \cdot \omega(t)\, dt$, *where the memory kernel f encodes how strongly the past influences the present. The* `whitenoise` *Python package implements this framework: it reads a two-column CSV, computes the empirical Mean Square Displacement (MSD), fits one of 18 theoretical MSD models, and extracts the memory parameter* μ *with 95% confidence intervals — producing publication-quality plots and summary tables in a single function call.*

How to use this guide: Follow the numbered sections in order during the workshop. All code blocks are self-contained and can be copied directly into a Python script or Jupyter notebook. The *Quick Reference Card* on the final page summarises all public API functions.

University of San Carlos · Center for Theoretical Physics and Allied Sciences (CTPAS) · CVIF, Philippines

Contents

1. The Full Pipeline
2. CSV Format
3. The Memory Parameter μ
4. Available Models
5. Model Selection
6. Optional Preprocessing
7. Normalization Note
8. The N Parameter
9. The Displacement PDF
10. Multi-dataset Comparison
11. Methodological Notes / Known Limitations
12. Quick Reference Card

INSTALLATION

Install the package and its dependencies with:

```
pip install whitenoise-swna
```

For Excel export support:

```
pip install whitenoise-swna[excel]
```

Import in Python:

```
import whitenoise as wn
import numpy as np
import matplotlib.pyplot as plt
```

CORE CONCEPT — WHAT SWNA MEASURES

SWNA fits a theoretical MSD model to your data's empirical Mean Square Displacement. The MSD measures how much a system's observable spreads over time:

$$MSD(\Delta) = \langle [x(i + \Delta) - x(i)]^2 \rangle$$

The memory parameter **μ** (or **H** for fBm) is extracted from the shape of this MSD curve. A value near $\mu = 1$ means near-Brownian (random walk) behaviour; $\mu > 1$ means the system's fluctuations are reinforced by its past; $\mu < 1$ means they are damped.

1. The Full Pipeline

The `whitenoise` package provides both a step-by-step manual workflow and a single-call shortcut. The table below maps each step to its corresponding function.

Step	Function	What it does
A	`wn.read_csv(path)`	Load a two-column CSV file. Returns `(time, values, metadata)`. Units and axis labels are extracted automatically from the header.
B	`wn.plot_series(time, values, metadata=meta)`	Plot the raw (or preprocessed) time series for visual inspection.
C	`wn.compute_msd(values)`	Compute the empirical MSD. Returns `(lags, msd)` arrays. Default max lag is `len(values)//2`.
D	`wn.plot_msd_empirical(lags, msd, metadata=meta)`	Scatter-plot the empirical MSD before fitting. Inspect the shape to choose a model.
E	`wn.fit_msd(lags, msd, model='...')`	Fit $N \cdot$ msd_model to the empirical MSD. Returns a `FitResult` with all parameters and 95% confidence intervals.
F	`fit.summary()`	Print the full parameter table: values, standard errors, and 95% CIs for every parameter.
G	matplotlib (manual)	Overlay the fitted theoretical MSD curve on the empirical scatter (see PDF section for code).
H	matplotlib (manual)	Plot the displacement histogram and compare it to the theoretical Gaussian PDF (see Section 9).
I	`wn.analyze(path, model='...')`	**One-call shortcut** that performs steps A through H automatically. Returns an `AnalysisResult`.

Full Pipeline Code

Steps A–H performed manually, followed by the equivalent one-liner (Step I):

```
import whitenoise as wn
import numpy as np
import matplotlib.pyplot as plt

# ── Steps A–H: manual workflow ──────────────────────────────────

# A. Load CSV
time, values, meta = wn.read_csv('mydata.csv')

# B. Plot raw time series
wn.plot_series(time, values, metadata=meta)

# C. Compute empirical MSD
lags, msd = wn.compute_msd(values)

# D. Inspect MSD shape before fitting
wn.plot_msd_empirical(lags, msd, metadata=meta)

# E. Fit a model (researcher chooses)
fit = wn.fit_msd(lags, msd, model='cosine')

# F. Print full parameter table
print(fit.summary())

# G. Overlay fitted curve on empirical MSD (manual)
plt.scatter(lags, msd, s=12, alpha=0.6, label='Empirical MSD')
plt.plot(lags, fit.msd_fitted, color='#C0392B', lw=2.5, label='Fit')
plt.legend(); plt.show()

# H. Displacement PDF — see Section 9 for full code

# ── Step I: one-call equivalent ─────────────────────────────────

result = wn.analyze('mydata.csv', model='cosine')
result.summary()                  # prints full parameter table
wn.plot_diagnostics(result)       # 2×2 figure: series, MSD, PDF, params
result.plot_all(show=True)        # 3-panel publication figure
```

WHEN TO USE THE MANUAL WORKFLOW VS. THE ONE-LINER

Use **Step I** (`wn.analyze()`) for a quick first look at any dataset. Use the **manual workflow (A–H)** when you need intermediate outputs — for example, to preprocess the data before fitting, to inspect the raw MSD shape to guide model selection, or to customise individual plots for a publication figure.

2. CSV Format

The package reads a two-column CSV with a structured header. The format is simple but exact. All axis labels and unit annotations on plots are derived automatically from this header — you never need to label axes manually.

Standard Format

```
time [months], sunspot_number [count]
1, 12.4
2, 15.1
3, 14.8
```

Format Rules

Rule	Detail
Row 1 is always the header	No exceptions. The first data row is row 2.
Column 1 = independent variable	Time, index, distance, wavelength, etc. Any independent variable is accepted — the column does not have to be named "time".
Column 2 = observable	The quantity being analysed (sunspot number, CO_2 concentration, flux, etc.).
Header format: `name [unit]`	Unit in square brackets. Space before bracket is optional: `time[months]` also works.
Unitless column	Use empty brackets: `flux []`. The axis label will be just `flux` (no parentheses).
Multi-column CSV	Supported for batch mode. Column 1 = time, all other columns = separate observables. Use `wn.read_csv_multi()` or `wn.batch_analyze()`.

Header Examples

Header string	Parsed name	Parsed unit	Axis label produced
`time [months]`	time	months	time (months)
`sunspot_number [count]`	sunspot_number	count	sunspot_number (count)
`flux []`	flux	(empty)	flux
`flux [unitless]`	flux	unitless	flux
`distance [km]`	distance	km	distance (km)
`co2[ppm]`	co2	ppm	co2 (ppm)

Multi-column CSV (Batch Mode)

```
time [yr], co2 [ppm], temperature [°C], sst [°C]
1958, 315.2, 14.1, 13.8
1959, 315.9, 14.0, 13.7
...
```

```
# Load multi-column CSV
datasets = wn.read_csv_multi('climate.csv')
# Returns list of (time, values, metadata) tuples — one per non-time column
print(len(datasets))   # 3  (co2, temperature, sst)

# Or send directly to batch analysis
results = wn.batch_analyze('climate.csv', model='cosine')
results.summary_table()
```

AUTOMATIC DATA VALIDATION WARNINGS

When reading a CSV, the package checks for common data problems and prints ⚠ warnings to the console. These warnings are *non-fatal* — data loading continues regardless. Watch for:

- **Non-monotonic time** — column 1 values are not strictly increasing.
- **Negative counts** — a column declared as "count" or "freq" contains negative values.
- **Possible column swap** — column 1 has an unusually large spread (std > 1000×mean), suggesting the observable may be in the wrong column.
- **Unrecognised time unit** — the unit string is not in the list of known time/index units (e.g. months, yr, days, s...). The package proceeds normally.

3. The Memory Parameter μ

The key output of every SWNA fit is **μ** (mu), the memory parameter. It quantifies how strongly past fluctuations influence the future evolution of the observable. Its value determines the diffusive regime of the system.

μ range	Regime	Physical meaning
`μ < 0.95`	**Subdiffusive**	Past fluctuations *resist* change — the observable spreads more slowly than a random walk. Typical in confined or damped systems.
`0.95 ≤ μ ≤ 1.05`	**Near-Brownian**	Weak or no memory — dynamics are close to an ordinary random walk. Fluctuations are essentially uncorrelated.
`1.05 < μ ≤ 2.0`	**Superdiffusive**	Past fluctuations *reinforce* future ones — the observable spreads faster than a random walk. Persistent or trending dynamics.
`μ > 2.0`	**Hyperballistic**	Extreme memory — very strongly persistent dynamics. Rare in natural systems; may indicate a driven or quasi-periodic process.

The fBm Special Case — Hurst Exponent H

For the Fractional Brownian Motion model (`fbm`), the Hurst exponent **H** replaces μ as the shape parameter. The MSD formula is simply T^{2H}.

H range	Regime	Notes
`H < 0.475`	**Subdiffusive**	Anti-persistent increments
`0.475 ≤ H ≤ 0.525`	**Near-Brownian**	H = 0.5 is pure Brownian motion
`H > 0.525`	**Superdiffusive**	Persistent increments

VALIDATED BENCHMARKS (PUBLISHED RESULTS)

These reference values give you a sense of typical μ ranges in real datasets:

System	Model	μ (published)	Reference
Philippine earthquakes	exponential	μ ≈ 1.00–1.19	Roque et al. 2024
Solar sunspots	exponential	μ ≈ 1.15	Toledo et al. 2024
GBR coral bleaching	cosine	μ ≈ 4.64	Elnar et al. 2021
CO_2 Keeling curve	cosine	μ ≈ 0.91–0.97	Elnar et al. 2024
X-ray binaries	cosine	μ ∈ [0.50, 1.39]	Calotes thesis 2024

Reading the Output

```
result = wn.analyze('sunspot.csv', model='exponential')
result.summary()

# Access parameters directly
print(result.mu)        # the memory parameter (μ or H for fbm)
print(result.regime)    # 'subdiffusive' | 'near-Brownian' | 'superdiffusive' | 'h
print(result.fit.params)             # {'mu': 1.15, 'beta': 0.08, 'N': 3.2}
print(result.fit.confidence_intervals)   # {'mu': (1.09, 1.21), ...}
print(result.fit.r_squared)          # e.g. 0.962
```

4. Available Models

The package implements a subset of the 18 MSD models tabulated in Bernido & Carpio-Bernido (2015), Table 2.1. The five models below are fully implemented and ready to use. The remaining models are registered but raise `NotImplementedError` — run `wn.list_models()` to see the current status of all models.

Model string	Params	MSD formula	Typical application
`'cosine'`	μ, ν	$\sqrt{\pi}\,\Gamma(\mu)\cos(\nu T/2)\,J_{\mu-1/2}(\nu T/2)\,(T/\nu)^{\mu-1/2}$	CO_2 Keeling curve, X-ray binaries, coral bleaching
`'exponential'`	μ, β	$\Gamma(\mu)\,\beta^{-\mu}\,T^{\mu-1}\,e^{-\beta/T}$	Solar sunspots, earthquake sequences
`'sine'`	μ, ν	Same as cosine but $\sin(\nu T/2)$ replaces $\cos(\nu T/2)$	Population dynamics (Tuñaco et al., unpublished)
`'fbm'`	H	T^{2H}	Self-similar random processes; H = 0.5 is ordinary Brownian motion
`'dna'`	a, b, c	$a - c \cdot e^{-bL}$	DNA nucleotide separations — plateau-shaped MSD, no μ parameter

Cosine vs. Sine — When to use which

Both models share the same functional form; only the oscillatory envelope differs. The **cosine** model is positive for lags $T < \pi/\nu$, then becomes unreliable (returns NaN beyond the first zero-crossing). The **sine** model starts from zero at $T = 0$ and stays positive for $T < 2\pi/\nu$, so it is useful when the MSD starts near zero and rises smoothly. In practice, try both and compare R^2 values.

DNA Model Note

The `dna` model describes *restricted diffusion* — the MSD rises from (a−c) at lag $L = 0$ and saturates at plateau value a as $L \to \infty$. It has no memory parameter μ; the plateau height a and the rate b are the physically meaningful outputs.

List All Models

```
wn.list_models()     # prints a formatted table of all 18 models and their status

# Get detailed info on a specific model
info = wn.get_model('cosine')
print(info['params'])       # ['mu', 'nu']
print(info['status'])       # 'available'
print(info['description'])  # 'Power-law memory with cosine modulation'
```

PARAMETER MEANINGS

Parameter	Symbol	Models	Meaning
Memory exponent	μ	cosine, exponential, sine	Controls diffusive regime. μ = 1 is Brownian.
Characteristic frequency	ν	cosine, sine	Sets the oscillation period of the MSD envelope.
Exponential rate	β	exponential	Controls the exponential decay modulation.
Hurst exponent	H	fbm	H = 0.5 is Brownian; H > 0.5 persistent.
Normalization scalar	N	all models	Scales MSD amplitude. Carries no physical meaning — see Section 8.

5. Model Selection

CRITICAL RULE — NO AUTOMATIC MODEL SELECTION

The package fits **exactly one model per `analyze()` call**. It never selects a model automatically. If $R^2 < 0.5$, the package will print a ⚠ warning suggesting alternatives, but it is the researcher's responsibility to re-run the analysis with a different model string.

R^2 Quality Thresholds

R^2 value	Interpretation	Action
`≥ 0.80`	Good fit	Trust the extracted parameters. Report with confidence.
`0.50 – 0.80`	Moderate fit	Parameters are usable but consider trying other models. Inspect the residuals.
`< 0.50`	Poor fit	The chosen model likely does not describe this data. Try a different model. The package will print a ⚠ warning automatically.

Manual Iteration Workflow

```
# Step 1 — Try exponential first (good starting point for many datasets)
result = wn.analyze('mydata.csv', model='exponential')
result.summary()   # read the R² value

# Step 2 — If R² < 0.8, try cosine
result2 = wn.analyze('mydata.csv', model='cosine')
result2.summary()

# Step 3 — Try fbm for self-similar or simple scaling datasets
result3 = wn.analyze('mydata.csv', model='fbm')
result3.summary()

# Step 4 — Search all available models at once
comp = wn.batch_model_search('mydata.csv')
wn.print_comparison(comp)        # side-by-side R² table
comp.summary_table()             # returns a pandas DataFrame
```

HOW TO READ THE EMPIRICAL MSD SHAPE TO GUIDE MODEL CHOICE

- **Monotone power-law rise:** Try `exponential` or `fbm`. The exponential model flattens at long lags; fBm remains a pure power law.
- **Oscillatory or peaked shape:** Try `cosine` or `sine`. The characteristic frequency ν will capture the period of the oscillation.
- **Plateau / saturation:** Try `dna`. This model is designed for MSD curves that level off at large lags.
- **Noisy or unclear shape:** Run `wn.batch_model_search()` and compare R^2 values across all models.

Inspecting MSD Shape Before Fitting

```
time, values, meta = wn.read_csv('mydata.csv')

# Optional: remove a deterministic trend first
fluctuations = wn.detrend(values, method='linear')

# Compute and plot MSD before choosing a model
lags, msd = wn.compute_msd(fluctuations)
wn.plot_msd_empirical(lags, msd, metadata=meta)
# → Look at the shape: rising power-law? oscillating? plateau?

# Then fit your chosen model
fit = wn.fit_msd(lags, msd, model='cosine')
print(fit.summary())
```

Using max_lag_fraction to Improve Fits

At large lags, the MSD estimate uses very few data pairs and becomes noisy. Restricting the fit to the first fraction of lags can improve parameter recovery:

```
# Fit using only the first 50% of lags (more reliable region)
result = wn.analyze('mydata.csv', model='cosine', max_lag_fraction=0.5)

# Or directly in fit_msd
fit = wn.fit_msd(lags, msd, model='cosine', max_lag_fraction=0.5)
print(f"Lags used: {len(fit.lags_used)} of {len(lags)}")
```

6. Optional Preprocessing

Preprocessing is entirely optional. The pipeline never applies it automatically. Use these functions when your data contains a deterministic trend that should be removed before computing the MSD, or when extreme data scale causes numerical instability.

Function	Method options	What it does
`wn.detrend(values, method='linear')`	`'linear'`, `'polynomial'`, `'mean'`	Subtract a polynomial trend from the data. Returns the residual fluctuations.
`wn.normalize(values, method='zscore')`	`'zscore'`, `'minmax'`, `'mean'`, `'maxabs'`	Rescale the data. The `N` parameter in the fit absorbs amplitude, so this is rarely needed.
`wn.smooth(values, window=5)`	`'moving_average'`, `'gaussian'`	Smooth the time series. Output has the same length as input.

Detrending

```
time, values, meta = wn.read_csv('co2.csv')

# Remove a linear trend
fluctuations = wn.detrend(values, method='linear')

# Or remove a quadratic trend
fluctuations = wn.detrend(values, method='polynomial', poly_order=2)

# Then plot and analyze the residuals
wn.plot_series(time, fluctuations, title='Detrended fluctuations')
result = wn.analyze(fluctuations, model='cosine', label='CO₂ fluctuations')
```

Comparing Before and After

```
import matplotlib.pyplot as plt

fluctuations = wn.detrend(values, method='linear')

fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
wn.plot_series(time, values,       metadata=meta,
               title='Original',           ax=axes[0], show=False)
wn.plot_series(time, fluctuations, metadata=meta,
               title='Detrended (linear)', ax=axes[1], show=False)
plt.tight_layout()
plt.show()
```

Normalization Methods

Method	Formula	Output range	When to use
`'zscore'`	(x – mean) / std	mean=0, std=1	General purpose standardisation
`'minmax'`	(x – min) / (max – min)	[0, 1]	When you need a bounded range
`'mean'`	x / mean	mean=1	Relative deviations from mean
`'maxabs'`	x / max(|x|)	[−1, 1]	Preserves zero and sign; useful for centred fluctuations

PREPROCESSING CHANGES YOUR DATA — DOCUMENT IT

If you detrend or smooth before calling `wn.analyze()`, the `AnalysisResult` will contain the preprocessed values — not the originals. Always document which preprocessing steps were applied when reporting results. The package does not track preprocessing history automatically.

7. Normalization Note

NORMALIZATION BEFORE ANALYSIS IS NOT STANDARD PRACTICE

In the Bernido group's published work, raw (or detrended) values are passed directly to `wn.analyze()` *without* prior normalization. This is correct by default.

The reason is that the fit always includes a normalization scalar **N**:

$$\mathrm{MSD}_{\mathrm{fitted}}(T) = N \cdot \mathrm{msd_model}(T, \mu, \ldots)$$

N absorbs the data amplitude. Scaling your data by a constant factor before fitting simply changes the fitted value of N — it does not affect μ, ν, β, or any other shape parameter.

When normalization might help: only in extreme edge cases where the raw data amplitude is so large (e.g. 10^{12}) that it causes numerical overflow or underflow in the MSD computation. In that case, `wn.normalize(values, method='maxabs')` is the recommended approach because it preserves zero and sign while scaling to [−1, 1].

Quick Demonstration

```
import whitenoise as wn
import numpy as np

np.random.seed(42)
data = np.random.randn(300) * 500     # large amplitude

# Without normalization (correct approach)
result1 = wn.analyze(data, model='cosine', label='Raw', verbose=False)

# With normalization
normalized = wn.normalize(data, method='maxabs')
result2 = wn.analyze(normalized, model='cosine', label='Normalized', verbose=False

# μ should be essentially identical; only N differs
print(f"μ (raw):        {result1.mu:.6f}")
print(f"μ (normalized): {result2.mu:.6f}")   # ≈ same
print(f"N (raw):        {result1.fit.params['N']:.4f}")
print(f"N (normalized): {result2.fit.params['N']:.4f}")   # different
```

8. The N Parameter

Every model fit includes a normalization scalar **N** alongside the physical parameters. Understanding its role prevents misinterpretation of the fit output.

What N does

The fitted MSD curve is always of the form:

$$\mathrm{MSD}_{\mathrm{fitted}}(T) = \mathbf{N} \cdot \mathrm{msd_model}(T, \mu, \ldots)$$

N scales the entire curve *vertically*. Changing N shifts the curve up or down without altering its shape in any way — and therefore without affecting the memory regime.

Parameter type	Parameters	Controls	Physically meaningful?
Shape parameters	μ, ν, β, H	The shape of the MSD curve — regime, exponent, frequency	Yes — these are the results to report
Scale parameter	N	The overall amplitude (height) of the MSD curve	No — absorbs data amplitude; not physically interpretable

> **WHAT TO REPORT IN PUBLICATIONS**
>
> Report μ (and ν or β as appropriate) with their 95% confidence intervals and the R^2 value. The N parameter is part of the fit output but carries no physical meaning and should not be reported as a result. It is included in `FitResult.params` for completeness and for constructing the fitted curve.

Accessing N in Code

```
result = wn.analyze('mydata.csv', model='cosine')

# Physical parameters (report these)
print(result.fit.params['mu'])    # memory parameter
print(result.fit.params['nu'])    # characteristic frequency
print(result.fit.confidence_intervals['mu'])   # 95% CI: (lo, hi)

# Scale parameter (do not report)
print(result.fit.params['N'])     # just for internal curve reconstruction
```

9. The Displacement PDF

SWNA predicts that the displacements $\Delta x(i) = x(i+T) - x(i)$ at any lag T follow a Gaussian distribution centred at zero with variance equal to the theoretical MSD:

$$P(\Delta x; T) = (2\pi\sigma^2)^{-1/2} \exp(-\Delta x^2 / (2\sigma^2)) \quad \text{where} \quad \sigma^2 = N \cdot \text{msd_model}(T, \mu, \ldots)$$

Comparing the empirical displacement histogram to this theoretical Gaussian is a key diagnostic: if the histogram matches well, the model and parameters are validated.

Automatic PDF via AnalysisResult

```
# The simplest way — uses a sensible default lag automatically
result = wn.analyze('mydata.csv', model='cosine')
result.plot_pdf()           # displacement histogram + theoretical Gaussian

# Or use the publish-quality version
wn.publish_pdf(result)      # same plot, publication styling
```

Manual PDF Construction (Full Control)

```
import whitenoise as wn
import numpy as np
import matplotlib.pyplot as plt

time, values, meta = wn.read_csv('mydata.csv')
lags, msd = wn.compute_msd(values)

MODEL = 'cosine'
fit = wn.fit_msd(lags, msd, model=MODEL)

# Choose a lag (short lags give more displacement pairs → better statistics)
lag_index = len(lags) // 10
T = int(lags[lag_index])

# Compute empirical displacements
displacements = values[T:] - values[:-T]

# Compute theoretical Gaussian variance
model_info = wn.get_model(MODEL)
phys_names = model_info['params']                    # e.g. ['mu', 'nu']
phys_vals  = [fit.params[p] for p in phys_names]
sigma2     = model_info['msd'](T, *phys_vals) * fit.params['N']
mu_disp    = np.mean(displacements)

# Plot
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.hist(displacements, bins='auto', density=True,
        color='#888888', alpha=0.55, label='Empirical displacements')

dx  = np.linspace(displacements.min(), displacements.max(), 500)
pdf = np.exp(-(dx - mu_disp)**2 / (2*sigma2)) / np.sqrt(2*np.pi*sigma2)
ax.plot(dx, pdf, color='#C0392B', lw=2.0, label=f'Theoretical PDF (T={T})')

ax.set_xlabel(f'Displacement Δx  ({meta.get("y_unit", "")})')
ax.set_ylabel('Probability Density')
ax.legend()
plt.show()
```

CHOOSING THE LAG T FOR THE PDF

- **Short lags** ($T \approx len/10$): many displacement pairs → smooth histogram. Use for a reliable PDF comparison.
- **Long lags**: fewer pairs → noisy histogram. The distribution should still be Gaussian but with larger scatter.
- The default in `wn.publish_pdf()` and `result.plot_pdf()` uses a model-appropriate lag: cosine/sine: $T = \pi/(2\nu)$; exponential: $T = len/10$; fBm: $T = len/4$.

10. Multi-dataset Comparison

The `compare()` and `batch_analyze()` functions let you run SWNA on multiple datasets in one call and compare memory parameters side by side.

compare() — Named Datasets

```
comparison = wn.compare({
    'Sunspots':     'exponential_sunspot.csv',
    'CO₂':          'cosine_co2.csv',
    'X-ray binary': 'cosine_xray.csv',
}, model='cosine')

# Print ASCII table
wn.print_comparison(comparison)

# Get as DataFrame
df = comparison.summary_table()
print(df[['label', 'mu', 'mu_ci_low', 'mu_ci_high', 'r_squared', 'regime']])

# Publication bar chart of μ with 95% CI error bars
wn.publish_comparison(comparison)

# Save to CSV
comparison.to_csv('results/comparison.csv')
```

Per-dataset Model Assignment

```
# Use a different model for each dataset
comparison = wn.compare(
    {
        'Sunspots':     'sunspot.csv',
        'CO₂':          'co2.csv',
    },
    model={
        'Sunspots': 'exponential',
        'CO₂':      'cosine',
    }
)
```

batch_analyze() — Three Input Modes

```
# Mode A: folder of CSVs
results = wn.batch_analyze('data/earthquakes/', model='exponential')

# Mode B: explicit list of paths
results = wn.batch_analyze(
    ['chile.csv', 'japan.csv', 'ph.csv'],
    model='exponential'
)

# Mode C: multi-column CSV (one column per system)
results = wn.batch_analyze('climate.csv', model='cosine')
# Each non-time column becomes a separate AnalysisResult

# All three return the same ComparisonResult type
print(len(results.results))        # number of systems analysed
results.summary_table()            # pandas DataFrame
results.to_csv('batch_out.csv')    # save to CSV
```

ComparisonResult Structure

Attribute / Method	Returns	Description
`.results`	list of AnalysisResult	One entry per dataset analysed
`.summary_table()`	pandas DataFrame	μ, CIs, R^2, regime — sorted by μ ascending
`.plot_comparison()`	Axes	Horizontal μ bar chart with CI error bars
`.to_csv(path)`	None	Save summary DataFrame to CSV
`.to_excel(path)`	None	Save to .xlsx (requires openpyxl)

11. Methodological Notes / Known Limitations

FITTING METHOD — UNWEIGHTED LEAST SQUARES

The current implementation fits the MSD model using **unweighted sum-of-squared residuals (ordinary least squares)**. This is consistent with the fitting approach used in the Bernido group's published results and ensures reproducibility of prior work.

More statistically rigorous alternative: Weighted Least Squares (WLS) with weights $w_\Delta = N - \Delta$ would be more principled, because large-lag MSD estimates are computed from fewer data pairs ($N - \Delta$ pairs for lag Δ) and are therefore noisier. Assigning lower weight to large-lag points would improve parameter accuracy. This is planned for a future release.

References for WLS rationale: Michalet, X. (2010). *Phys. Rev. E* 82, 041914. Vestergaard, C. L., Blainey, P. C., & Flyvbjerg, H. (2014). *Phys. Rev. E* 89, 022726.

STUB MODELS — NOT YET IMPLEMENTED

Of the 18 models in the registry, **9 are stubs** that raise `NotImplementedError` when called. Only the following are currently available:

```
cosine    exponential    sine    fbm    dna    sin_half    cos_half
inc_gamma    bessel_j0_cos    bessel_jmu_nu
```

Run `wn.list_models()` to see the current status of all models. Attempting to use a stub model will produce a clear error message listing the available alternatives.

OTHER KNOWN LIMITATIONS

- **Batch parallel mode:** When `n_jobs > 1`, all CSV paths must be accessible from disk. In-memory numpy arrays cannot be passed to parallel workers — use `n_jobs=1` in that case.
- **matplotlib backend:** `import whitenoise as wn` does not import matplotlib at package level. On headless servers (no display), set the backend before calling any viz function: `import matplotlib; matplotlib.use("Agg")`
- **Cosine model validity:** The cosine MSD formula becomes negative (and returns NaN) for lags $T > \pi/\nu$. If your MSD fit returns NaN over a large lag range, reduce `max_lag_fraction` or switch to the `sine` or `exponential` model.
- **NaN handling:** Isolated NaN values in the input are linearly interpolated. If more than 50% of the input is NaN, a clear error is raised.

ARCHITECTURE NOTE — INDEPENDENT VARIABLE COLUMN

Column 1 of the CSV is the *independent variable* — it does not have to be labelled "time". It can be distance, wavelength, genomic position, sequence index, or any other quantity. The validation rules still apply (monotone ordering, known unit names), but the column name is freely chosen.

12. Quick Reference Card

All names below are importable directly from `whitenoise` (i.e. `wn.<name>`).

I/O

Function	Description
`wn.read_csv(path)`	Load 2-column whitenoise-format CSV → (time, values, metadata)
`wn.read_csv_multi(path)`	Load multi-column CSV → list of (time, values, metadata) tuples
`wn.export_csv(result, path)`	Save AnalysisResult or ComparisonResult to CSV
`wn.export_summary(cr, path)`	Save ComparisonResult summary DataFrame to CSV

Core

Function	Description
`wn.compute_msd(values)`	Compute empirical MSD → (lags, msd) arrays
`wn.list_models()`	Print table of all 18 SWNA models and their status
`wn.get_model(name)`	Return model info dict (msd function, params, status)
`wn.fit_msd(lags, msd, model)`	Fit N · msd_model to empirical MSD → FitResult or None
`fit.summary()`	Print formatted parameter table with 95% CIs and R^2

Preprocessing (optional)

Function	Description
`wn.detrend(values, method='linear')`	Subtract trend ('linear', 'polynomial', 'mean') → residuals
`wn.normalize(values, method='zscore')`	Normalize ('zscore', 'minmax', 'mean', 'maxabs')
`wn.smooth(values, window=5)`	Smooth ('moving_average', 'gaussian'); output same length as input

Analysis

Function	Description
`wn.analyze(path, model)`	Full SWNA pipeline on a CSV or array → AnalysisResult
`result.summary()`	Print full parameter table and regime interpretation
`result.mu`	Memory parameter (μ for all models; H for fbm)
`result.regime`	Diffusive regime string (e.g. 'superdiffusive')
`result.to_csv(path)`	Save single-row result CSV
`wn.compare(datasets, model)`	Analyze multiple datasets → ComparisonResult
`wn.print_comparison(cr)`	Print ASCII μ comparison table
`wn.batch_analyze(source, model)`	Batch analysis (folder / list / multi-CSV) → ComparisonResult
`wn.batch_model_search(path)`	Try all available models on one file → ComparisonResult

Exploratory Plots (viz/explore.py)

Function	Description
`wn.plot_series(time, values, metadata=meta)`	Raw time series line plot with zero reference and grid
`wn.plot_msd_empirical(lags, msd, metadata=meta)`	MSD scatter with Brownian linear reference line
`wn.plot_diagnostics(result)`	2×2 figure: time series, MSD, PDF, parameter text box
`wn.plot_msd(result)`	Empirical MSD scatter + fitted theoretical curve
`wn.plot_pdf(result)`	Displacement histogram + theoretical Gaussian PDF
`wn.plot_timeseries(result)`	Time series from AnalysisResult (uses stored metadata)

Publication Plots (viz/publish.py)

Function	Description
`wn.publish_msd(result)`	MSD plot: serif font, inward ticks, dpi=150, no top/right spines
`wn.publish_pdf(result)`	PDF plot with publication styling
`wn.publish_comparison(cr)`	μ horizontal bar chart with 95% CI error bars, colour by regime

Common kwargs (all viz functions)

kwarg	Default	Description
`ax=None`	None	Pass an existing Axes to embed the plot in a larger figure
`show=True`	True	Set `show=False` to suppress `plt.show()` (e.g. when saving to file)

CITATION

When using results from this package in publications, please cite:

Bernido, C. C., & Carpio-Bernido, M. V. (2015). *Stochastic and Infinite Dimensional Analysis*. Trends in Mathematics. Birkhäuser, Basel. DOI: 10.1007/978-3-319-07245-6

Contact: Center for Theoretical Physics and Allied Sciences (CTPAS), University of San Carlos, Cebu City, Philippines.